TECH BIZ SECURITY · SOC ANALYST INTERNSHIP · BATCH B-503

WEEK 2 · TASK 6

Understanding Logs & Log Analysis Basics

Submitted by	**Oscar ALIDJINOU**
Student ID	**TBS-INT-2026-2398**
Programme	**SOC Analyst Internship (6 weeks)**
Company	**Tech Biz Security**
Date of submission	**29-10-2026**

01 Objective

An analyst rarely needs more data, they need to know how to read the data they already have. This task covered the foundations of log analysis: the main log types and where they come from, how to read a web-server log line, how to tell a normal entry from a suspicious one, and what a baseline is and why it matters. I tested each idea in my own Kali Linux lab by generating real traffic against my own machine, and I completed the TryHackMe room Intro to Log Analysis.

02 Proof of Completion

Room	Duration	Tasks	Status
TryHackMe: Intro to Logs	30 min	7	Completed in my Week 2 Task 4 report
TryHackMe: Investigating Windows	45 min	1	Completed in my Week 2 Task 5 report
TryHackMe: Intro to Log Analysis	60 min	10	Completed, room progress 100%

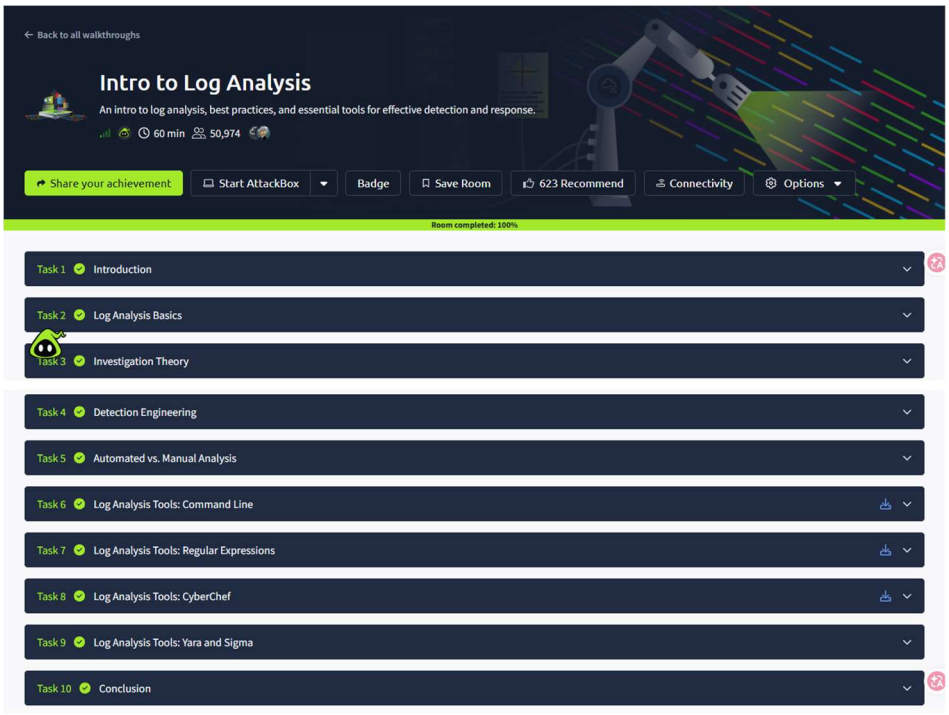


Figure 1. Intro to Log Analysis: all 10 tasks completed (100%).

> **My lab**
> All practical work was done on a Kali Linux virtual machine. Every scan and test request in this report was aimed only at my own machine (127.0.0.1). Scanning or attacking a system I do not own or have permission to test is illegal and against the rules of this programme.

03 Log Types and Where They Come From

Different systems write different kinds of logs. These are the five types I need to recognise first, with the matching evidence from my own /var/log folder.

Log type	What it records	Where it comes from	Example in my lab
System	Kernel messages, boot sequence, service starts and stops, hardware status.	The operating system and its services (kernel, systemd).	`/var/log/syslog` `kern.log, boot.log` `journalctl`
Security (authentication)	Logins, failed logins, sudo use, privilege changes.	sshd, sudo and the PAM login system.	`/var/log/auth.log`
Application	Messages from one program: status, warnings, errors.	The application itself.	`/var/log/apt/term.log` `apache2/error.log`
Network	Connections, firewall decisions, VPN and network device events.	Firewalls, routers, IDS/IPS, VPN software.	`/var/log/ufw.log` `openvpn/ folder`
Web server	Every HTTP request: client IP, time, request, status, user agent.	Apache or Nginx.	`/var/log/apache2/access.log`

Other types that appeared in the room and in my lab are database logs (the mysql and postgresql folders) and audit logs, which record who did what and when.

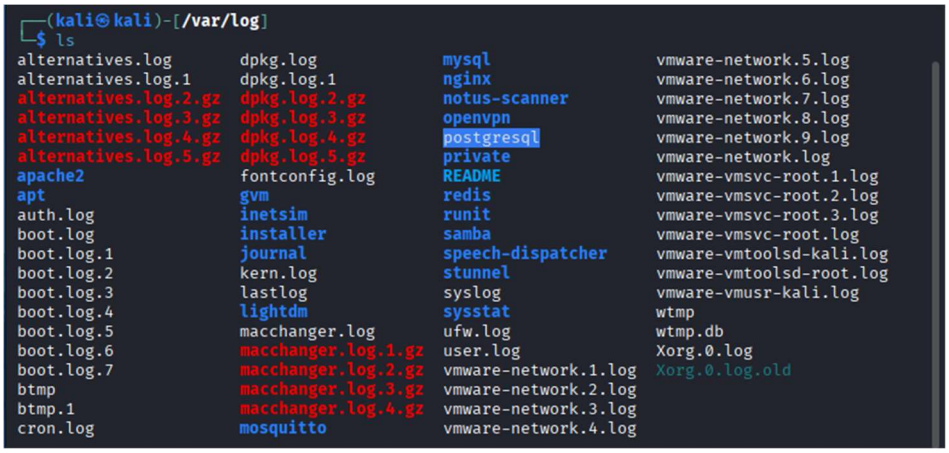


Figure 2. My /var/log folder: system, authentication, web server, database and firewall logs side by side.

Seeing the types in action

- **Security:** I tried to log in over SSH to a user that does not exist (faux) and typed wrong passwords. I then followed auth.log live with tail -f, which showed my own sudo activity.
- **System:** I restarted Apache and read the system journal. It showed systemd stopping the old Apache processes and starting the service again, all within about three seconds.
- **Boot:** boot.log lists each service that started during startup, including the logrotate timer that rotates logs.

```
┌──(kali㉿kali)-[~]
└─$ ssh faux@127.0.0.1
faux@127.0.0.1's password:
hbh
Permission denied, please try again.
faux@127.0.0.1's password:
Permission denied, please try again.
faux@127.0.0.1's password:
```

```
┌──(kali㉿kali)-[~]
└─$ sudo tail -n 3 -f /var/log/auth.log
2026-10-29T01:10:19.746097+01:00 kali sudo: pam_unix(sudo:session): session closed for user r
oot
2026-10-29T01:10:25.983329+01:00 kali sudo:     kali : TTY=pts/0 ; PWD=/home/kali ; USER=root
 ; COMMAND=/usr/bin/tail -n 3 -f /var/log/auth.log
2026-10-29T01:10:25.985263+01:00 kali sudo: pam_unix(sudo:session): session opened for user r
oot(uid=0) by kali(uid=1000)
```

Figure 3. A failed SSH login attempt, and auth.log followed live with tail -f.

```
┌──(kali㉿kali)-[~]
└─$ sudo journalctl -n 3 -f
oct. 29 01:12:09 kali sudo[5046]:     kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND
=/usr/bin/journalctl -n 3 -f
oct. 29 01:12:09 kali sudo[5046]: pam_unix(sudo:session): session opened for user root(uid=0)
 by kali(uid=1000)
oct. 29 01:12:14 kali sudo[5050]:     kali : TTY=pts/1 ; PWD=/home/kali ; USER=root ; COMMAND
=/usr/bin/systemctl restart apache2
oct. 29 01:12:14 kali sudo[5050]: pam_unix(sudo:session): session opened for user root(uid=0)
 by kali(uid=1000)
oct. 29 01:12:14 kali systemd[1]: Stopping apache2.service - The Apache HTTP Server...
oct. 29 01:12:15 kali apachectl[5055]: AH00558: apache2: Could not reliably determine the ser
ver's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally t
o suppress this message
oct. 29 01:12:16 kali systemd[1]: apache2.service: Killing process 3763 (apache2) with signal
 SIGKILL.
oct. 29 01:12:16 kali systemd[1]: apache2.service: Killing process 3764 (apache2) with signal
 SIGKILL.
oct. 29 01:12:16 kali systemd[1]: apache2.service: Killing process 3765 (apache2) with signal
 SIGKILL.
oct. 29 01:12:16 kali systemd[1]: apache2.service: Killing process 3767 (apache2) with signal
 SIGKILL.
oct. 29 01:12:16 kali systemd[1]: apache2.service: Deactivated successfully.
oct. 29 01:12:16 kali systemd[1]: Stopped apache2.service - The Apache HTTP Server.
oct. 29 01:12:16 kali systemd[1]: apache2.service: Consumed 6.038s CPU time over 32min 40.153
s wall clock time, 77.1M memory peak, 14.2M memory swap peak.
oct. 29 01:12:16 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
oct. 29 01:12:16 kali apachectl[5062]: AH00558: apache2: Could not reliably determine the ser
ver's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally t
```

Figure 4. journalctl: systemd stopping and restarting apache2.

```
┌──(kali㉿kali)-[/var/log]
└─$ sudo cat boot.log
──────────── Thu Oct 29 16:03:58 CET 2026 ────────────
/dev/sda1: clean, 890613/4980736 files, 11187670/19922944 blocks
[  OK  ] Finished plymouth-read-write.service - Tell Plymouth To Write Out Runtime Data.
[  OK  ] Finished console-setup.service - Set console font and keymap.
[  OK  ] Finished systemd-userdb-load-credentials.service - Load JSON user/group Records from
 Credentials.
[  OK  ] Finished systemd-tmpfiles-setup.service - Create System Files and Directories.
         Mounting run-rpc_pipefs.mount - RPC Pipe File System...
[  OK  ] Started haveged.service - Entropy Daemon based on the HAVEGE algorithm.
         Starting open-vm-tools.service - Service for virtual machines hosted on VMware...
[  OK  ] Reached target sysinit.target - System Initialization.
[  OK  ] Started apt-daily.timer - Daily apt download activities.
[  OK  ] Started apt-daily-upgrade.timer - Daily apt upgrade and clean activities.
[  OK  ] Started dpkg-db-backup.timer - Daily dpkg database backup timer.
[  OK  ] Started e2scrub_all.timer - Periodic ext4 Online Metadata Check for All Filesystems.
[  OK  ] Started fstrim.timer - Discard unused filesystem blocks once a week.
[  OK  ] Started logrotate.timer - Daily rotation of log files.
[  OK  ] Started man-db.timer - Daily man-db regeneration.
[  OK  ] Started phpsessionclean.timer - Clean PHP session files every 30 mins.
[  OK  ] Started plocate-updatedb.timer - Update the plocate database daily.
[  OK  ] Started systemd-tmpfiles-clean.timer - Daily Cleanup of Temporary Directories.
[  OK  ] Reached target paths.target - Path Units.
```

Figure 5. boot.log: services started during boot.

04 Breaking Down an Apache Log Line

To get real Apache logs, I restarted the web server and requested its default page with curl, then read the access and error logs. This is the first line in my access log:

```
127.0.0.1 - - [29/Oct/2026:16:54:55 +0100] "HEAD / HTTP/1.1" 200 255 "-" "curl/8.20.0"
```

Field	Value in my line	What it tells an analyst
Client IP	127.0.0.1	Who made the request. Here it is my own machine; in real logs this is the address to investigate.
Identity and user	- -	Two unused fields, shown as dashes (remote identity and logged-in user).
Time	[29/Oct/2026:16:54:55 +0100]	Date and time of the request. +0100 is the time zone offset, important for building timelines.
Request	"HEAD / HTTP/1.1"	The method (HEAD), the resource (/) and the protocol version.
Status	200	The server's answer: 200 means success.
Size	255	Size of the response in bytes.
Referer	"-"	The page that linked here; empty here.
User agent	"curl/8.20.0"	The client software, here the curl tool rather than a browser.

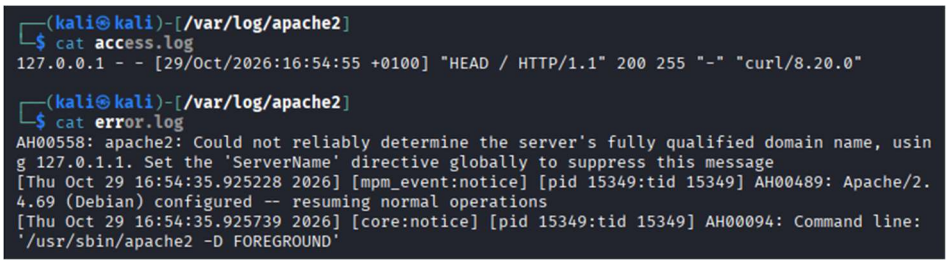


Figure 6. access.log and error.log after restarting Apache.

The error log is different: it records the server itself. After my restart it showed a notice that Apache could not determine its fully qualified domain name (code AH00558, a harmless configuration warning) and a notice that Apache was configured and resuming normal operations.

To make long lines easier to read, I used awk to split each access log line into labelled fields:



Figure 7. awk turning raw access log lines into IP, time, request and status.

> **Status code groups**
> 2xx means success, 3xx a redirect, 4xx a client error (such as 404 not found or 400 bad request), and 5xx a server error. A sudden wave of 4xx responses from one IP is a classic sign of scanning.

05 Normal vs Suspicious Log Entries

A normal entry is one that fits what the system is used for. To compare, I created suspicious traffic against my own Apache server: a path traversal request, an SQL injection attempt, and a scan with Nikto, a free web vulnerability scanner.

```
┌──(kali㉿kali)-[/var/log/apache2]
└─$ curl "http://127.0.0.1/products.php?id=1' OR '1'='1"
curl: (3) URL rejected: Malformed input to a URL function

┌──(kali㉿kali)-[/var/log/apache2]
└─$ curl --path-as-is "http://127.0.0.1/../../etc/passwd"
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>400 Bad Request</title>
</head><body>
<h1>Bad Request</h1>
<p>Your browser sent a request that this server could not understand.<br />
</p>
<hr>
<address>Apache/2.4.69 (Debian) Server at 127.0.0.1 Port 80</address>
</body></html>
```

Figure 8. A SQL injection attempt (rejected by curl) and a path traversal request (answered 400 Bad Request).

```
┌──(kali㉿kali)-[/var/log/apache2]
└─$ nikto -h http://127.0.0.1
- Nikto v2.6.0
---------------------------------------------------------------------------
+ Target IP:          127.0.0.1
+ Target Hostname:    127.0.0.1
+ Target Port:        80
+ Platform:           Linux/Unix
+ Start Time:         2026-10-29 17:05:16 (GMT1)
---------------------------------------------------------------------------
+ Server: Apache/2.4.69 (Debian)
+ ERROR: Failed to check for updates: 403
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped
.
+ [999984] /: Server may leak inodes via ETags, header found with file /, inode: 29cf, size:
65efcb09f4002, mtime: gzip. See: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-1418
+ [013587] /: Suggested security header missing: content-security-policy. See: https://develo
per.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.m
ozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
```

Figure 9. Running Nikto against my own web server.

The scan produced thousands of lines. To find the dangerous ones, I filtered the access log with grep for known attack strings:

```
sudo grep -Ei "OR%20|etc/passwd|nikto|sqlmap" /var/log/apache2/access.log
```

```
┌──(kali㉿kali)-[/var/log/apache2]
└─$ sudo grep -Ei "OR%20|etc/passwd|nikto|sqlmap" /var/log/apache2/access.log
127.0.0.1 - - [29/Oct/2026:17:05:00 +0100] "GET /../../etc/passwd HTTP/1.1" 400 523 "-" "curl
/8.20.0"
127.0.0.1 - - [29/Oct/2026:17:05:16 +0100] "PUT /nikto-test-JzSgY4cZ.html HTTP/1.1" 405 590 "
-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/1
42.0.6281.32 Safari/537.36 Edg/142.0.6281.32"
127.0.0.1 - - [29/Oct/2026:17:05:20 +0100] "GET /cfdocs/exampleapp/email/getfile.cfm?fileNikt
o=/etc/hosts HTTP/1.1" 404 527 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537
.36 (KHTML, like Gecko) Chrome/142.0.6284.18 Safari/537.36 Brave/1.67.120"
127.0.0.1 - - [29/Oct/2026:17:05:20 +0100] "GET /autohtml.php?op=modload&mainfile=x&Nikto=/et
c/hosts HTTP/1.1" 404 527 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (
KHTML, like Gecko) Chrome/142.0.6284.18 Safari/537.36 Brave/1.67.120"
127.0.0.1 - - [29/Oct/2026:17:05:20 +0100] "GET /site/'%20UNION%20ALL%20SELECT%20FileToClob('
/etc/hosts','server')::html,0%20FROM%20sysusers%20WHERE%20userNikto=USER%20\\-\\-\\/\\.html H
TTP/1.1" 404 527 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 14_4) AppleWebKit/605.1.15 (KHTM
L, like Gecko) Version/17.4 Safari/605.1.15"
127.0.0.1 - - [29/Oct/2026:17:05:20 +0100] "GET /site/'%20UNION%20ALL%20SELECT%20FileToClob('
/etc/hosts','server')::html,0%20FROM%20sysusers%20WHERE%20userNikto%20=%20USER%20\\-\\-\\/\\.
html HTTP/1.1" 404 527 "-" "Mozilla/5.0 (Linux; Android 14; Pixel 8) AppleWebKit/537.36 (KHTM
L, like Gecko) Chrome/141.0.6252.82 Mobile Safari/537.36"
127.0.0.1 - - [29/Oct/2026:17:05:20 +0100] "GET /user.php?op=confirmnewuser&module=NS-NewUser
&uNikto=%22%3E%3Cimg%20src=%22javascript:alert(document.cookie);%22%3E&email=test@test.com HT
```

Figure 10. Isolating the suspicious lines from the access log.

Indicator	Normal entry	Suspicious entry
Example	"GET /index.html HTTP/1.1" 200	"GET /../../etc/passwd HTTP/1.1" 400
Resource requested	A real page of the site.	Paths that climb out of the web folder or target system files (/etc/passwd, /etc/hosts).
Status	200	A flood of 404, 400 and 405 responses.
Content of the request	Plain page names.	Attack payloads such as UNION ALL SELECT (SQL injection) and <img ...alert(document.cookie)> (cross-site scripting).
Speed	A request now and then.	Many requests within the same second.
User agent	One consistent client.	Browser names that change from line to line, including Windows, Mac and Android, all from one IP.
Tool fingerprints	None.	The scanner's name inside the requests (PUT /nikto-test-... returned 405).

> **Analyst assessment**
> These lines are a web vulnerability scan combining path traversal, SQL injection and cross-site scripting probes. The server refused them, answering 400, 404 and 405. On a real server I would block the source IP, check that no probe returned 200, and raise an alert.

A suspicious SSH pattern

I also simulated repeated failed SSH logins against a fake account on my own machine, using a loop of ten attempts. A normal user mistypes a password once or twice; a loop like this produces a burst of failures in seconds.

```
┌──(kali㉿kali)-[/var/log/apache2]
└─$ for i in {1..10}; do
  ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no \
      -o ConnectTimeout=3 faux@127.0.0.1 true
done
faux@127.0.0.1's password:
dds
dd
ddd
sPermission denied, please try again.
faux@127.0.0.1's password:
Permission denied, please try again.
faux@127.0.0.1's password:
faux@127.0.0.1: Permission denied (publickey,password).
faux@127.0.0.1's password:
Permission denied, please try again.
faux@127.0.0.1's password:
Permission denied, please try again.
faux@127.0.0.1's password:
faux@127.0.0.1: Permission denied (publickey,password).
faux@127.0.0.1's password:
```

Figure 11. Repeated failed SSH attempts for the fake user faux, on localhost.

My lab timeline

Time (+01:00)	What I did, and what the log shows
16:54	Restarted Apache. First request with curl: HEAD / answered 200.
17:00 - 17:01	Normal requests: GET /index.html, 200.
17:05:00	Path traversal request: GET /../../etc/passwd, answered 400.
17:05:16	First Nikto scan starts; thousands of requests within seconds.

Time (+01:00)	What I did, and what the log shows
17:12:18	Second Nikto scan starts.
After 17:12	Loop of failed SSH logins for the fake user faux.

06 Baselines

6.1 What a baseline is

A baseline is a record of what normal looks like on a system: how many requests it gets, from whom, with which status codes, and how many failed logins it usually has. Without one, I cannot tell whether 500 requests an hour is quiet or an attack. With one, anything far from normal stands out, which is how anomalies are spotted.

6.2 How I measured it

I wrote a short shell script that counts four things in the logs: requests per IP, requests per status code, requests per hour, and failed SSH passwords. I ran it once for the "normal" baseline and again after the second scan and the SSH attempts, saving each result to its own file.

```
┌──(kali㉿kali)-[/var/log/apache2]
└─$ L=/var/log/apache2/access.log
{
  echo "== IP =="; sudo awk '{print $1}' $L | sort | uniq -c | sort -rn
  echo "== Statuts =="; sudo awk '{print $9}' $L | sort | uniq -c
  echo "== Requêtes/heure =="; sudo awk '{print substr($4,14,2)}' $L | sort | uniq -c
  echo "== Échecs SSH =="; sudo grep -c "Failed password" /var/log/auth.log
} > ~/baseline_normal.txt
```

Figure 12. The script that builds the baseline report (saved to baseline_normal.txt).

I then compared the two files with diff:

```
┌──(kali㉿kali)-[/var/log/apache2]
└─$ diff ~/baseline_normal.txt ~/baseline_attaque.txt
2c2
<    7186 127.0.0.1
---
>   14358 127.0.0.1
4,10c4,10
<     131 200
<      82 400
<       3 403
<    6958 404
<       3 405
<       7 501
<       2 523
---
>     255 200
>     157 400
>       6 403
>   13916 404
>       6 405
>      14 501
>       4 523
13c13
<    7185 17
---
>   14357 17
15c15
< 8
---
> 14
```

Figure 13. diff of the two baseline files: the numbers on the right are after the scan.

Measure	Baseline	After scan and SSH attempts	Change
Requests from 127.0.0.1	7,186	14,358	+7,172 (about double)
Requests in hour 17	7,185	14,357	+7,172
Status 200	131	255	+124
Status 400	82	157	+75
Status 404	6,958	13,916	+6,958 (exactly double)
Failed SSH passwords	8	14	+6

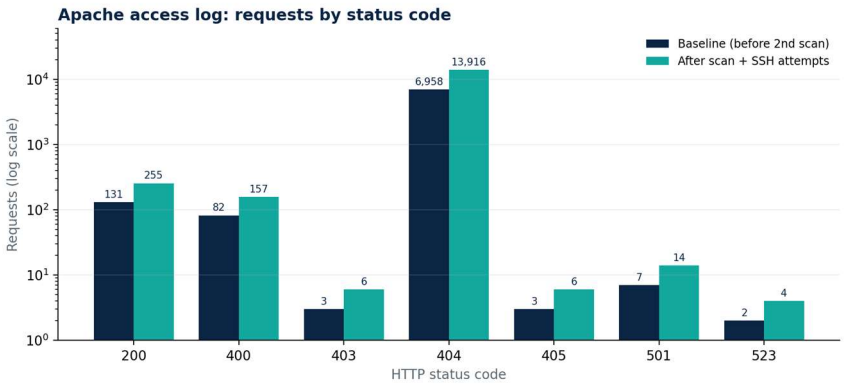


Figure 14. Requests by status code before and after the second scan (log scale).

6.3 What the comparison shows

- Every count that Nikto touches roughly doubled, because the second scan repeated the same requests. The 404 count doubled exactly, which is the signature of an automated scan.
- The number of failed SSH passwords rose by six, matching the failed attempts I made.
- All the extra traffic came from one address and fell within the same hour. A rule such as "more than a few hundred 404 responses from one IP in an hour" would catch it.

> **A limitation I noticed**
> My "normal" baseline was taken after my first Nikto scan, so it already held about 7,000 scan requests, and that is why its 404 count was so high. A true baseline must be captured before any test traffic and over a representative period, such as several days. I would do it in that order next time.

07 Intro to Log Analysis (TryHackMe)

Task	Topic	What I learned
2	Log analysis basics	A log entry has a timestamp, a source and details. The main log types are application, audit, security, server, system, network, database and web server logs.
3	Investigation theory	A timeline puts events in order and shows how an incident unfolded. Timestamps and time zones must be consistent. Data visualisation, monitoring and alerting, and threat intelligence help to understand and act on the data.

Task	Topic	What I learned
4	Detection engineering	Knowing common log locations (for example /var/log/nginx/access.log and /var/log/auth.log) and common attack patterns: SQL injection, cross-site scripting and path traversal.
5	Automated vs manual	Automated tools process large volumes quickly; manual analysis uses human judgement for unusual cases. A good analyst uses both.
6	Command line	cat, less, tail, head, wc, cut, sort, uniq, sed, awk and grep.
7	Regular expressions	Patterns that match text, used with grep and for parsing logs.
8	CyberChef	A browser tool that decodes, extracts and transforms data.
9	Sigma and Yara	Pattern-matching rule formats for detecting events in logs and files.

7.1 Command-line analysis

Command	What it does	How I used it
`cut -d ' ' -f 7`	Extracts one column using a delimiter.	Listed the requested pages from a sample Apache log.
`grep -c`	Counts matching lines.	Counted the requests answered with status 200.
`sort | uniq -c | sort -nr`	Counts and ranks identical lines.	Ranked IP addresses by number of requests.
`grep ... | grep ...`	Chains two searches to narrow results.	Found the requests from one IP to the login page.

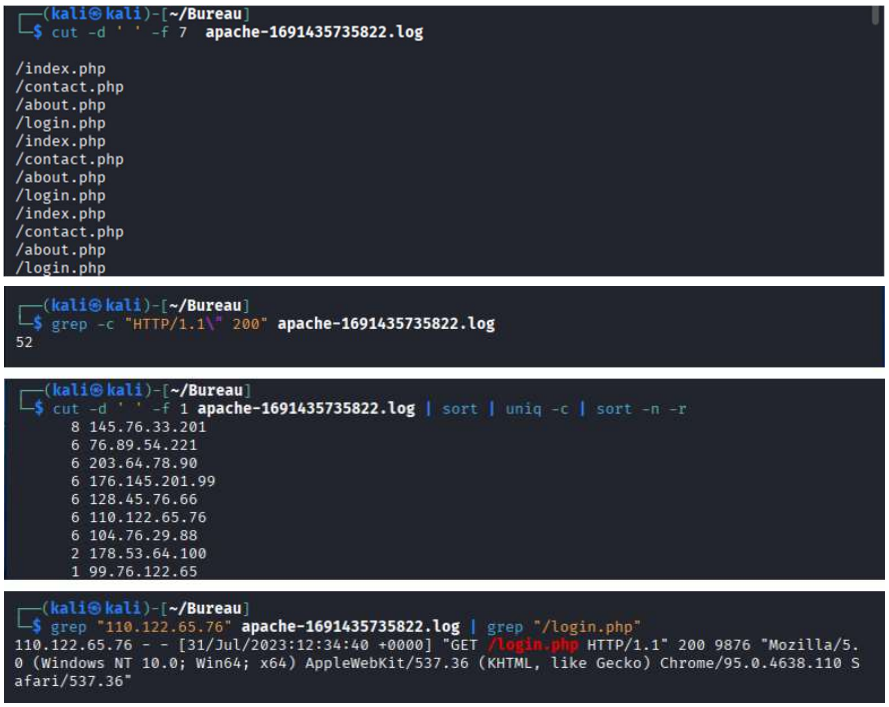


Figure 15. Analysing a sample Apache log: extracting pages, counting 200 responses, ranking IPs and tracing one visitor.

7.2 Regular expressions with grep

With grep -E, a character class such as [0-9] matches one digit. The pattern post=1[0-9] found blog requests for posts 10 to 19, and post=2[0-9] found posts 20 to 29.

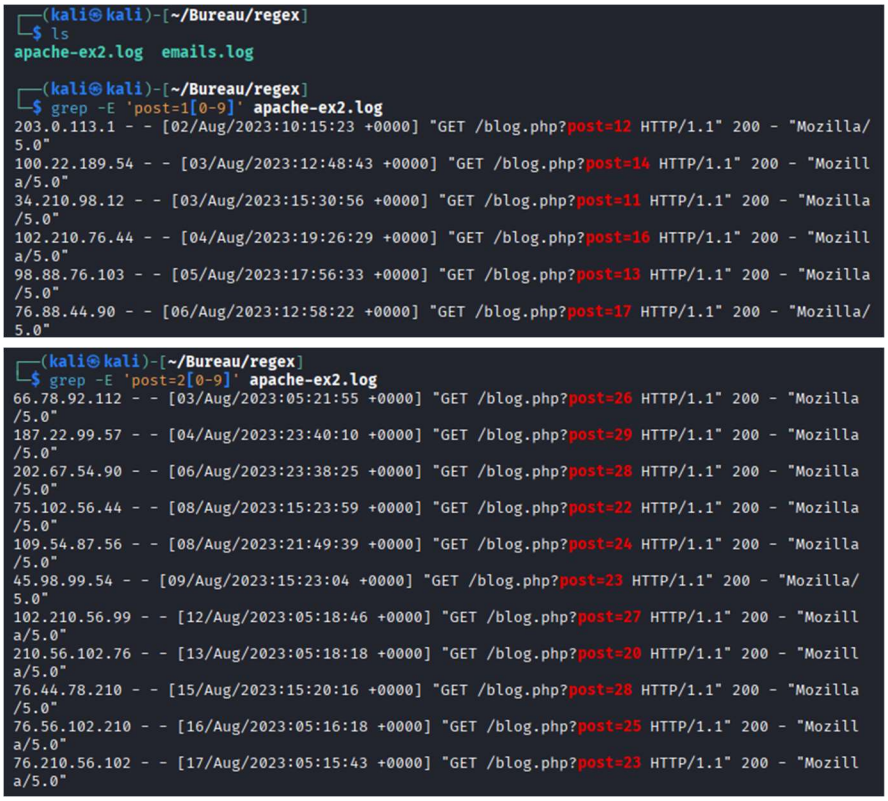


Figure 16. grep -E with a digit range on a sample log.

7.3 CyberChef

CyberChef builds a recipe of operations. I used its Regular expression operation with the built-in patterns to pull every IPv4 address out of a raw access log, and every MAC address out of an encoded file. It can also decode Base64 data, which attackers often use to hide commands.

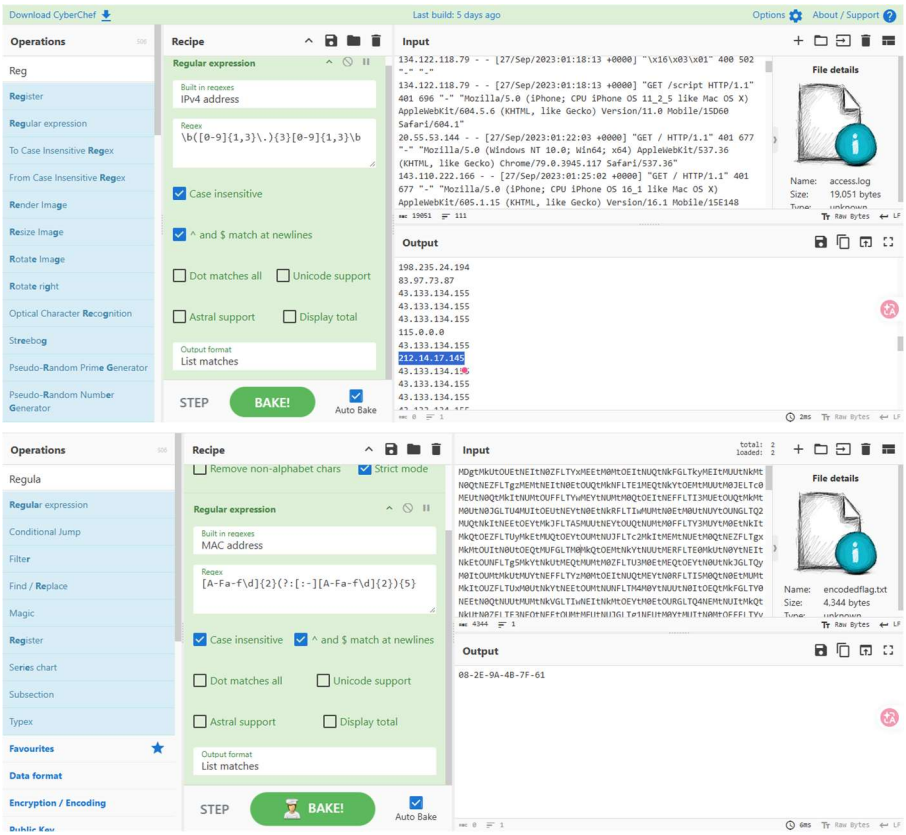


Figure 17. CyberChef extracting IPv4 addresses and a MAC address with regular expressions.

7.4 Sigma and Yara

	Sigma	Yara
Used for	Describing events to find in log files; converted into SIEM searches.	Matching text or binary patterns in files and data; mostly used in malware analysis.
Main keys	title, description, status, author, logsource, detection, falsepositives.	rule name, meta (author), strings, condition.
Example in the room	A rule named Failed SSH Logins that looks in sshd logs for lines containing "Failed" or "Illegal". It lists mistyped passwords as a false positive.	A rule named IPFinder that uses a regular expression for IPv4 addresses, and triggers when one is found.

08 The Defender's View

Attack signatures to recognise in web logs

Attack	What it looks like in the log	Seen in my lab
Path traversal	Sequences such as ../ and requests for sensitive files like /etc/passwd or /etc/shadow.	GET /../../etc/passwd, answered 400.
SQL injection	Unusual SQL in the URL: OR '1'='1', UNION SELECT.	UNION ALL SELECT probes from the scan.
Cross-site scripting	Script tags and event handlers (<script>, onerror, onclick) in the request.	A request carrying alert(document.cookie).

Attack	What it looks like in the log	Seen in my lab
Scanning	Hundreds or thousands of requests in seconds, mostly 404, from one IP.	About 7,000 requests per Nikto run.
Password guessing	Many failed login lines in a short time.	My loop of failed SSH logins.

Detections I would set up

- An alert when one IP causes more than a set number of 404 responses in a minute.
- An alert on any request containing ../ or /etc/passwd.
- An alert when failed SSH logins from one address exceed a threshold, using the baseline to choose it.
- A daily baseline comparison of requests per hour and status codes, to spot sudden jumps.

09 Reflection and Key Takeaways

- Knowing which log holds which evidence is the first skill of an analyst: authentication in auth.log, web requests in access.log, system events in the journal.
- A suspicious entry only stands out against a normal one, which is why baselines matter. Mine also taught me that a baseline can be spoiled if test traffic is already present.
- Generating my own attack traffic, safely and only against my own machine, showed me exactly how scans and probes look in the logs.
- Command-line tools plus regular expressions let me turn thousands of lines into a clear answer in seconds.

Next step: build a clean baseline over several days and write my first Sigma rule for the web attack patterns above.

10 References

- TryHackMe: Intro to Log Analysis, Intro to Logs and Investigating Windows (free rooms)
- Apache HTTP Server documentation: log files
- Sigma and Yara project documentation